CE 502 · ADVANCED TOPICS IN DATABASE SYSTEMS

Week 03

Concurrency Control & ACID

What happens when many users change the same data at the same time, and how a DBMS keeps their transactions from damaging each other.

THIS WEEK

- 3-1 **Concurrency Problems** — 45 minutes
- 3-2 **Locking and Deadlocks** — 45 minutes
- 3-3 **ACID and Isolation Levels** — 45 minutes
- P3 **Practice: race yourself** — 60 minutes · PostgreSQL lab

Dr. Afan HASAN · [Semester]

Recap: Protecting Data Against Failures

- **Backups** plus archived logs let a DBA restore a database to any moment, even after a disk or site is lost.
- **Replication** ships the log to a standby server, which can take over within seconds.
- **RPO** and **RTO** state how much data may be lost and how long recovery may take.

THIS WEEK: A NEW THREAT

1. Nothing fails, but **many users** change the same data at once.
2. The DBMS must keep their transactions **isolated**.

Key point: Weeks 1–2 delivered atomicity and durability. Week 3 adds isolation, and with it the full meaning of ACID.

CE 502 · WEEK 3 · LESSON

3-1

Concurrency Problems

Lesson 1 of 3

In this lesson

01 Why transactions run concurrently

02 Four classic concurrency problems

03 Tracing a schedule step by step

04 Serializability: when interleaving is safe

45 minutes

Builds on Weeks 1–2: transactions and the log

Objectives

By the end of this lesson, you will be able to:

01 **Explain** what concurrent transactions are and why a DBMS interleaves them

Checked in: Quiz 3 · question 1

02 **Describe** the lost update, dirty read, non-repeatable read and phantom read

Checked in: Quiz 3 · questions 2–3

03 **Trace** an interleaved schedule and identify the problem it causes

Checked in: Quiz 3 · question 4 and Practice 3

04 **Explain** when an interleaved schedule is serializable

Checked in: Quiz 3 · question 5

The objective chip in the top-right corner of each slide shows which goal that slide serves.

Objective 01

What Are Concurrent Transactions?

- **Concurrent transactions** run against the same database at the same time, so their operations overlap in time.
- The DBMS **interleaves** their steps: it switches between transactions, especially while one of them waits for the disk.
- On a multi-core server, transactions also run truly in parallel, so the overlap is real, not only apparent.

Two transactions, A and B

Serial: one after the other



Interleaved: steps alternate



While A waits for the disk or the network, B uses the CPU, so both finish sooner.

Key point: without concurrency, every user would wait for every other user; a busy system could not work.

Objective 01

Running transactions at the same time is easy. Running them correctly is the hard part.

If two transactions touch different data, any interleaving is safe. Problems appear only when they use the same data item and at least one of them writes it.

- Rule of thumb: a conflict needs two transactions, the same item, and at least one write.

Four Classic Concurrency Problems

Lost update

Two transactions read the same value and both write it back; the second write **overwrites** the first.

Dirty read

A transaction reads a change that another transaction has **not committed** and may still roll back.

Non-repeatable read

The same row, read twice in one transaction, gives **different values** because another transaction committed an update.

Phantom read

The same query, run twice in one transaction, returns a **different set of rows** because another transaction committed an insert or delete.

A fifth problem, the **deadlock**, is caused by the cure itself, locking. It is the topic of Lesson 3-2.

Example: Two Updates Without a Conflict

User A updates stock item 500; user B updates stock item 800. Their steps are interleaved.

Step	Processing order in the DBMS	Item
1	Read item 500 for A into the cache	500
2	Read item 800 for B into the cache	800
3	Update item 500 for A (in the cache)	500
4	Update item 800 for B (in the cache)	800
5	Commit A	500
6	Commit B	800

Why it works: A and B never touch the same item, so the order of their steps does not matter.

- Any interleaving of these six steps gives the same final database.

Key point: interleaving is dangerous only when transactions share a data item; most transactions in a busy system never do.

Example: The Lost Update

Item 500 has count = 120. A adds 12 and B subtracts 10. The correct result is 122.

Step	User A (+12)	User B (−10)	Stored count
1	Reads item 500: 120		120
2		Reads item 500: 120	120
3	Computes 120 + 12 = 132		120
4	Writes 132 and commits		132
5		Computes 120 − 10 = 110	132
6		Writes 110 and commits	110

← **Critical point:** B computes from a stale 120. A's +12 is lost, yet both commits succeed.

No error is reported. The data is simply wrong.

⚠ **Common mistake:** assuming that a successful COMMIT means the data is right. Both commits here succeeded; the result is still wrong.

Example: The Dirty Read

T2 changes a price but has not committed. T1 reads the new price.

Step	T1: invoicing	T2: price update
1		BEGIN; sets price of item 800 from 40 to 400 (a typo)
2	Reads price of item 800: 400	
3	Creates an invoice for 400	
4		Notices the typo: ROLLBACK (price is 40 again)
5	Commits the invoice for 400	

Why it matters: T1 acted on a value that was never committed and no longer exists.

⚠ **Common mistake:** thinking a dirty read leaves bad data in the table. The table is correct after the rollback; the damage is in T1's decision.

Objective 02

Non-Repeatable Reads and Phantom Reads

Non-repeatable read

```
T1: SELECT bal WHERE id = 1 → 100
T2: UPDATE bal = 500; COMMIT
T1: SELECT bal WHERE id = 1 → 500
```

The **same row** has a different value the second time.

Phantom read

```
T1: SELECT count(*) WHERE bal ≥ 100 → 2
T2: INSERT (3, 300); COMMIT
T1: SELECT count(*) WHERE bal ≥ 100 → 3
```

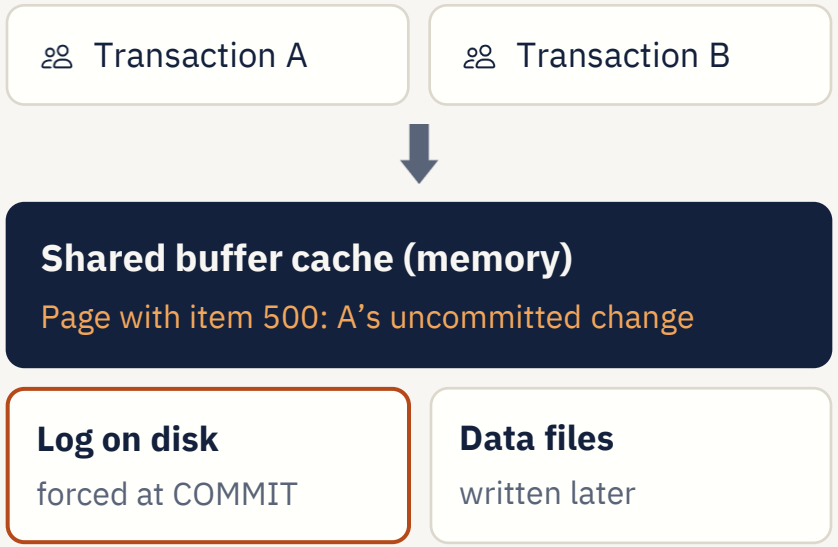
The **same query** returns a different set of rows.

⚠ **Common mistake:** calling every inconsistent read “dirty”. Here T1 only ever sees **committed** data; it changed between the two reads.

Objective 02

Where Do Uncommitted Changes Live?

- Transactions change data pages in the **shared buffer cache**, an area of memory that every transaction uses.
- At **COMMIT** the DBMS forces the transaction's **log records** to disk; changed data pages are written later (Week 1).
- So other transactions could physically reach uncommitted data. **Concurrency control** decides what they may see and change.



⚠ **Common mistake:** "updates are written to the database on disk when COMMIT is issued". It is the **log** that is forced at COMMIT.

Objective 04

When Is an Interleaved Schedule Correct?

A schedule is **serializable** if its result equals the result of running the same transactions one after another, in some order.

Order of execution	Final count	Serializable?
Serial: all of A, then all of B	122	Yes (serial)
Serial: all of B, then all of A	122	Yes (serial)
Interleaved, no conflicting overlap	122	Yes
Interleaved: the lost-update schedule	110	No

← **Read this row first.** 110 matches no serial order, so the schedule is not serializable.

Key point: the DBMS does not have to run transactions one at a time. It must only guarantee a result that some serial order could have produced.

Objective 03

Case Study: The Last Concert Ticket

THE SITUATION

A concert hall has 1,000 seats. The booking program reads **seats_left**; if it is above 0, it writes seats_left – 1 and creates a ticket.

At 10:00:00, two fans click “Buy” for the last seat within 5 milliseconds.

The next morning the manager says:

> “Our report shows **1,001 tickets** sold for a 1,000-seat hall.”

YOUR TASK · PAIRS · 6 MIN

1. Write the schedule that produced 1,001 tickets.
2. Which concurrency problem is this?
3. What would any serial order have produced?
4. Suggest one way to prevent it.

Objective 02

Check Your Understanding

T1 reads the balance of account 1 and gets 100. T2 updates the balance to 500 and commits. T1 reads the same balance again, in the same transaction, and gets 500. Which problem is this?

A Dirty read

B Lost update

C Non-repeatable read

D Phantom read

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Concurrent transactions

Transactions whose operations overlap in time on the same database.

Schedule

The order in which the DBMS executes the operations of several transactions.

Dirty read

Reading a change that another transaction has not committed.

Phantom read

The same query, run twice in a transaction, returns a different set of rows.

Interleaving

Executing the steps of several transactions alternately.

Lost update

A committed write is overwritten by a transaction that read the old value.

Non-repeatable read

The same row, read twice in a transaction, has different values.

Serializable schedule

A schedule whose result equals that of some serial order.

Summary

In this lesson, you learned how to:

- **01** Explain what concurrent transactions are and why a DBMS interleaves them
- **02** Describe the lost update, dirty read, non-repeatable read and phantom read
- **03** Trace an interleaved schedule and identify the problem it causes
- **04** Explain when an interleaved schedule is serializable

Key takeaway: transactions may overlap freely, but the result must be one that some serial order could have produced.

Next lesson

3-2 Locking and Deadlocks

CE 502 · WEEK 3 · LESSON

3-2

Locking and Deadlocks

Lesson 2 of 3

In this lesson

01 How locks prevent conflicts

02 Shared and exclusive locks; lock granularity

03 Two-phase locking

04 Deadlocks: detection, resolution and prevention

45 minutes

Builds on Lesson 3-1: the four problems

Objectives

By the end of this lesson, you will be able to:

01 **Distinguish** shared and exclusive locks, and implicit and explicit locks

Checked in: Quiz 3 · question 6

02 **Compare** lock granularities and the trade-off between concurrency and overhead

Checked in: Quiz 3 · question 7

03 **Explain** how two-phase locking produces serializable schedules

Checked in: Quiz 3 · question 8

04 **Explain** how deadlocks arise and how a DBMS detects, resolves and helps prevent them

Checked in: Quiz 3 · questions 9–10 and Practice 3

The objective chip in the top-right corner of each slide shows which goal that slide serves.

Objective 01

How Does a DBMS Prevent Conflicts?

- **Resource locking**: before a transaction changes an item, it locks it, and conflicting transactions must **wait**.
- **Implicit** locks are placed automatically by the DBMS for every INSERT, UPDATE and DELETE.
- **Explicit** locks are requested by the application with SQL commands; they are rarely needed.

```sql
-- implicit: the DBMS locks the row itself
UPDATE stock SET count = count - 10 WHERE item = 500;
-- explicit row lock, requested by the application
SELECT count FROM stock WHERE item = 500 FOR UPDATE;
-- explicit table lock: rarely needed
LOCK TABLE stock IN EXCLUSIVE MODE;
```

⚠ **Common mistake:** adding explicit locks “to be safe”. They lock more data for longer and increase the risk of deadlocks.

Shared and Exclusive Locks

- A **shared** (S) lock allows other transactions to **read** the item, but not to change it.
- An **exclusive** (X) lock allows no other transaction to read it with a lock or to change it.
- Readers take shared locks; writers take exclusive locks, held until the transaction ends.

Lock compatibility

Lock held ↓ / requested →	Shared	Exclusive
Shared	Granted	Waits
Exclusive	Waits	Waits

← **Read this cell first.** Only shared + shared is compatible: readers never block each other.

In practice

With MVCC (Lesson 3-3), ordinary reads in PostgreSQL and Oracle take **no** shared row locks; only writers lock rows. SELECT … FOR SHARE requests a shared row lock explicitly.

Objective 02

Lock Granularity: How Much to Lock?

Lock granularity is the size of the data item that one lock protects.

Lock covers	Concurrency	Overhead	Typical use
One row	Highest	Highest	OLTP: orders, payments
One page	High	Medium	Some older engines
One table	Low	Low	Bulk loads, schema changes
Whole database	Lowest	Lowest	Maintenance, restore

In practice

PostgreSQL stores row locks in the row itself, so it never escalates. SQL Server **escalates** to a table lock when one statement holds about 5,000 locks on one object.

Key point: small locks let more transactions work at once; large locks cost less to manage. Every DBMS balances the two.

Objective 03

Example: Locking Prevents the Lost Update

The same two users as in Lesson 3-1, but each locks item 500 before reading it.

Step	User A (+12)	User B (−10)	Count
1	Locks item 500, reads 120		120
2		Requests the lock: waits	120
3	Writes 132, commits, releases	(waiting)	132
4		Gets the lock, reads 132	132
5		Writes 122, commits, releases	122

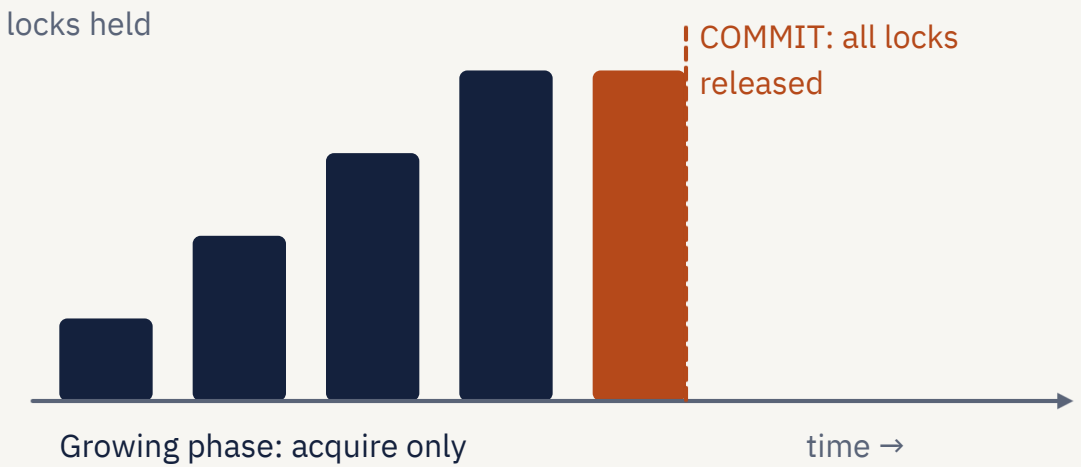
Critical point: B waits instead of reading a stale value, so it computes from 132.

The result, 122, is the same as running A, then B.

Objective 03

Two-Phase Locking (2PL)

- **Growing phase:** the transaction acquires locks and releases none.
- **Shrinking phase:** once it releases any lock, it may not acquire another.
- If every transaction follows 2PL, every schedule is **serializable**.
- **Strict 2PL** holds exclusive locks until COMMIT or ROLLBACK, so nobody reads uncommitted data.



Key point: almost every lock-based DBMS uses strict 2PL: locks pile up during the transaction and are all released at the end.

Objective 04

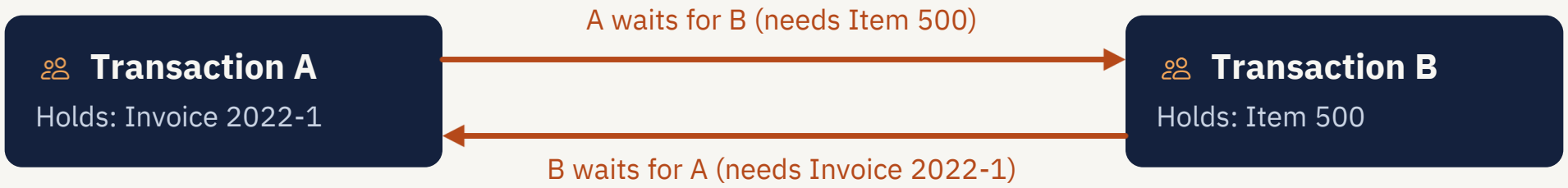
Locking turns conflicts into waiting. A deadlock is waiting that never ends.

Two transactions each hold a lock that the other one needs. Neither can continue, and neither will release its lock until it finishes.

- Rule of thumb: every lock-based system has deadlocks; good systems make them rare and handle them.

Objective 04

What a Deadlock Looks Like: the Wait-For Graph



- Each arrow means “waits for a lock held by”. A **cycle** of arrows is a deadlock.

Critical point: no one in the cycle can move, so the DBMS must break it.

Objective 04

Example: A Deadlock, Step by Step

Step	What happens	Waits for
1	A locks and reads invoice 2022-1	–
2	B locks and reads item 500	–
3	A requests item 500	A → B
4	B requests invoice 2022-1: a cycle	B → A
5	The DBMS detects the deadlock and aborts B	–
6	B is rolled back; its lock on item 500 is released	–
7	A gets item 500, updates both and commits	–

Critical point: the victim is chosen by a rule or by cost, not at random.

B's application receives an error and must **retry** the whole transaction.

Key point: the victim loses its work, not its data: the log rolls it back cleanly, exactly as in Week 1.

Objective 04

How a DBMS Detects and Breaks a Deadlock

01

A long wait

A lock wait lasts longer than a threshold: **deadlock_timeout**, 1 s by default in PostgreSQL.

02

Look for a cycle

The DBMS follows the wait-for graph from the waiting transaction.

03

Choose a victim

By rule or cost: priority, work to undo, or who triggered the check.

04

Abort and retry

The victim is rolled back, its locks released; the application retries it.

Key point: deadlock errors are normal events in a busy system. Applications must catch them and retry the transaction.

Objective 04

How to Make Deadlocks Rare

Same order

Access shared tables and rows in one agreed order, e.g. by ascending key.

Short transactions

No user think time or network calls while locks are held.

Fail fast

Set **lock_timeout** so a stuck wait ends with an error that can be retried.

Lock less

Avoid unneeded explicit locks; index foreign keys so updates lock fewer rows.

⚠ **Common mistake:** adding hardware. A deadlock is a cycle in the logic; a faster server only reaches the cycle sooner.

Objective 04

Case Study: The Nightly Deadlock

THE SITUATION

Two nightly jobs run at 02:00. **Job 1** (invoicing) updates a customer row, then that customer's orders. **Job 2** (refunds) updates an order, then its customer row.

About once a week, one job fails with "deadlock detected" and the night's work is incomplete.

> "We added more memory to the server and it still happens."

YOUR TASK · PAIRS · 7 MIN

1. Draw the wait-for graph of one deadlock.
2. Why did more memory not help?
3. What happens to the victim job, and what should it do?
4. Give two changes that make the deadlock rare.

Objective 04

Check Your Understanding

Transaction A holds an exclusive lock on row 1 and requests row 2. Transaction B holds an exclusive lock on row 2 and requests row 1. What does PostgreSQL do?

A Both wait until an administrator ends one of them

B After deadlock_timeout it finds the cycle and aborts one; that one must be retried

C It merges the two transactions into one

D It grants both requests, because they are on different rows

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Lock

A marker that controls which transactions may read or change an item.

Exclusive lock

Lets no other transaction lock or change the item.

Lock escalation

Replacing many small locks with one larger lock to save resources.

Deadlock

Transactions each waiting for a lock that another one in the group holds.

Shared lock

Lets others read the item but not change it.

Lock granularity

The size of the item one lock protects: row, page, table or database.

Two-phase locking

No lock is acquired after any lock has been released.

Wait-for graph

A graph of which transaction waits for which; a cycle means deadlock.

Summary

In this lesson, you learned how to:

- **01** Distinguish shared and exclusive locks, and implicit and explicit locks
- **02** Compare lock granularities and their trade-off
- **03** Explain how two-phase locking produces serializable schedules
- **04** Explain how deadlocks arise and how a DBMS detects, resolves and helps prevent them

Key takeaway: locks force conflicting transactions to wait; two-phase locking makes the result serializable, and deadlocks are handled by aborting and retrying.

Next lesson

3-3 ACID and Isolation Levels

CE 502 · WEEK 3 · LESSON

3-3

ACID and Isolation Levels

Lesson 3 of 3

In this lesson

01 The four ACID properties and their mechanisms

02 The SQL isolation levels and their anomalies

03 MVCC and snapshot isolation in PostgreSQL

04 Choosing a level and handling aborts

45 minutes

Builds on Lessons 3-1 and 3-2

Objectives

By the end of this lesson, you will be able to:

01 **Define** atomicity, consistency, isolation and durability, and the mechanism behind each

Checked in: Quiz 3 · question 11

02 **Distinguish** the four SQL isolation levels by the anomalies they allow

Checked in: Quiz 3 · questions 12–13 and Practice 3

03 **Explain** how MVCC and snapshot isolation work, and how PostgreSQL implements the levels

Checked in: Quiz 3 · questions 14–15

04 **Decide** on an isolation level and error handling for an application

Checked in: Quiz 3 · question 16 and the design question

The objective chip in the top-right corner of each slide shows which goal that slide serves.

Objective 01

The Four ACID Properties

A Atomicity

All changes of a transaction are kept, or none are.

Mechanism: the log, used to undo a failed transaction (Week 1)

C Consistency

A transaction takes the database from one valid state to another.

Mechanism: constraints in the DBMS, plus correct application logic

I Isolation

Concurrent transactions do not see each other's unfinished work.

Mechanism: locking or MVCC; strength set by the isolation level

D Durability

Once committed, changes survive crashes.

Mechanism: the log forced to disk at COMMIT; backups (Weeks 1–2)

⚠ **Common mistake:** "durable = data written to disk at COMMIT". The **log** is forced at COMMIT; data pages are written later.

Objective 01

Consistency: Whose Job Is It?

The DBMS enforces

- Primary and unique keys
- Foreign keys
- NOT NULL and CHECK constraints
- Triggers that you define

</> The application must ensure

- Business rules the DBMS does not know, e.g. “a transfer keeps the total unchanged”
- The right SQL inside one transaction
- The right isolation level for its rules

Key point: declare every rule you can as a constraint. The DBMS then checks it for every transaction, at every isolation level.

The Four SQL Isolation Levels

The SQL standard (SQL-92 onwards) defines each level by the anomalies it must prevent.

Isolation level	Dirty read	Non-repeatable read	Phantom read
Read Uncommitted	Possible	Possible	Possible
Read Committed	Prevented	Possible	Possible
Repeatable Read	Prevented	Prevented	Possible
Serializable	Prevented	Prevented	Prevented

Read the diagonal. Each level up removes one more anomaly; only Serializable removes all.

In practice

The default is **Read Committed** in PostgreSQL, Oracle and SQL Server, and **Repeatable Read** in MySQL InnoDB. Set it per transaction: BEGIN ISOLATION LEVEL REPEATABLE READ;

Objective 02

Why Choose a Lower Level?



- Stronger isolation means more locks held for longer, or more transactions **aborted** and retried.
- Weaker isolation means more concurrency, but the application must tolerate or prevent the anomalies.

⚠ **Common mistake:** assuming the default level protects you. Read Committed allows lost updates in read-compute-write code.

Multiversion Concurrency Control (MVCC)

- An UPDATE creates a **new version** of the row instead of overwriting it.
- Each transaction reads from a **snapshot**: the versions committed when the snapshot was taken.
- Readers never block writers, and writers never block readers. Writers of the **same row** still wait for each other.

Row: account 1

Version 1 · bal = 100
created by T10 (committed)

Version 2 · bal = 500
created by T12 (not yet committed)

T11 started before T12 committed, so its snapshot shows **version 1**, without waiting.

In practice

PostgreSQL, Oracle, MySQL InnoDB and SQL Server (with snapshot options) use MVCC. In PostgreSQL, **VACUUM** removes old versions no snapshot needs.

How PostgreSQL Implements the Levels

Level requested	What PostgreSQL does	Anomalies left
Read Uncommitted	Behaves exactly like Read Committed	Non-repeatable read, phantom
Read Committed	New snapshot for each statement	Non-repeatable read, phantom, lost update
Repeatable Read	One snapshot for the whole transaction (snapshot isolation)	Write skew only
Serializable	Snapshot isolation + conflict detection (SSI)	None

Note: at Repeatable Read PostgreSQL also prevents phantoms, which the standard allows.

Key point: at Repeatable Read and Serializable, PostgreSQL may abort a transaction with **could not serialize access** (SQLSTATE 40001). The application must retry.

Write Skew: What Snapshot Isolation Allows

Both transactions run at Repeatable Read in PostgreSQL. Each one is correct on its own.

Step	T1: Dr Ana	T2: Dr Ben
0	Rule: at least one doctor on call. On call now: Ana, Ben	
1	Counts doctors on call: 2	
2		Counts doctors on call: 2
3	Sets Ana off call; commits	
4		Sets Ben off call; commits
5	Result: nobody is on call	

← **Why?** They update different rows, so no write conflict is seen. Only Serializable (SSI) aborts one.

Key point: snapshot isolation stops two writes to the **same row**. It does not see a conflict between what one transaction reads and another writes.

Objective 04

Every level below Serializable is a performance option with conditions attached.

Lower levels are faster and abort less, but they allow anomalies. They are safe only if you know which anomalies your transactions can suffer and prevent those in another way.

Rule of thumb: keep the default, find the transactions that can be harmed, and protect those explicitly.

Objective 04

Handling Aborts: Retry the Whole Transaction

```
-- Python (psycopg 3)
for attempt in range(5):
    try:
        with conn.transaction(): book_seat(conn)
        break
    except (errors.SerializationFailure, errors.DeadlockDetected):
        time.sleep(random.uniform(0, 0.05 * 2 ** attempt))
```

1 Try at most five times.

3 Run the whole transaction; COMMIT or ROLLBACK is automatic.

← **5 · Critical:** catch only **40001** and **40P01**. Other errors are real bugs.

6 Wait a short, growing, random time.

⚠ **Common mistake:** retrying only the failed statement. After an abort the whole transaction is rolled back; its earlier reads are no longer valid.

Choosing an Isolation Level for Each Transaction

Decide per transaction, not once for the whole database.

Transaction	Main risk	A good choice
Transfer between two accounts	Lost update, write skew	Serializable + retry, or FOR UPDATE in key order
Book the last seat	Lost update	Read Committed + atomic UPDATE … WHERE seats_left > 0
Month-end report over many tables	Inconsistent totals	Repeatable Read, read-only
Live dashboard of approximate counts	None that matters	Read Committed (default)

Read this column last. Often the best fix is better SQL, not a higher level.

Key point: protect the transactions that can be harmed; leave the rest at the default level.

Objective 04

ACID Is a Spectrum, Not a Divide

- Traditional relational DBMSs provide full ACID transactions on one server.
- Many NoSQL systems chose weaker guarantees, such as single-item atomicity and **eventual consistency**, for scale and availability.
- The line has blurred: many NoSQL systems now offer transactions, and distributed SQL systems offer ACID across servers.

In practice

MongoDB: multi-document ACID transactions since 4.0 (2018).
Cassandra: tunable consistency and lightweight transactions.
Spanner, CockroachDB: serializable transactions across regions.

Key point: ask what a product guarantees for your transactions, not whether it is “SQL” or “NoSQL”. Week 9 returns to this question.

Objective 04

Case Study: Course Registration

THE SITUATION

A university registration system runs at the default Read Committed level. Courses have a capacity; students enrol online, and a nightly report counts enrolments per department.

After the first week of term, the registrar writes:

> “A course with 30 places has **32 students**, and the department totals in the report do not add up.”

YOUR TASK · PAIRS · 8 MIN

1. Which anomaly explains each complaint?
2. How would you fix enrolment?
3. Which level should the report use, and why?
4. What must the application do if the DBMS aborts a transaction?

Check Your Understanding

Which is the lowest isolation level in the SQL standard that prevents non-repeatable reads, but may still allow phantom reads?

A Read Uncommitted

B Read Committed

C Repeatable Read

D Serializable

Think for 30 seconds, then vote with your fingers: 1 = A, 2 = B, 3 = C, 4 = D.

Key Terms

Atomicity

All changes of a transaction are kept, or none are.

Consistency

A transaction takes the database from one valid state to another.

Isolation

Concurrent transactions do not see each other’s unfinished work.

Durability

Committed changes survive any later failure.

Isolation level

A setting that chooses which anomalies a transaction may see.

MVCC

Keeping several row versions so readers use a snapshot without waiting.

Snapshot isolation

Every read in a transaction uses one snapshot taken at its start.

Write skew

Two transactions read the same data, write different rows, and break a rule.

Summary

In this lesson, you learned how to:

- **01** Define atomicity, consistency, isolation and durability, and the mechanism behind each
- **02** Distinguish the four SQL isolation levels by the anomalies they allow
- **03** Explain how MVCC and snapshot isolation work, and how PostgreSQL implements the levels
- **04** Decide on an isolation level and error handling for an application

Key takeaway: isolation is the one ACID property you can tune: choose the protection each transaction needs, and always be ready to retry.

Next

Practice 3: Race Yourself

Practice 3: Two Sessions, One Database

You run two transactions against each other by hand and watch what the DBMS does.

Setup	Two psql sessions side by side: **A** on the left, **B** on the right. Table **acc** with two rows.	–
Part A	Four experiments: deadlock, non-repeatable read, lost update and phantom, at two isolation levels.	Lessons 3-1 to 3-3
Part B	A lost update in Python code with two threads, then two fixes.	Lessons 3-1, 3-2
Report	Your anomaly table, explanations and a retry strategy.	Lesson 3-3

You need: Docker, the course PostgreSQL 16 container, Python 3.10+ with psycopg, and the **labs/week03** folder of the course kit.

Part A · Setup and the Four Experiments

```
-- in BOTH terminals
1 docker exec -it ce502-postgres psql -U postgres -d lab
2 \i /kit/labs/week03/sql/00_setup.sql -- reset before each experiment
3 -- then run labs/week03/sql/e1_…e4_/session_a.sql and session_b.sql step by step
```

Experiment	Run at	What you should see
E1 Deadlock	Read Committed	One session gets deadlock detected
E2 Non-repeatable read	RC, then RR	Second read: 500, then 100
E3 Lost update	RC, then RR	150 (one +50 lost), then an error
E4 Phantom	RC, then RR	Count 2 → 3, then 2 → 2

← **Critical:** run one step at a time, in step order, switching terminals as the files say.

Part A · Inside Experiment E3: The Lost Update

```
-- E3 · session A and session B
1 A, B: BEGIN ISOLATION LEVEL READ COMMITTED;
2 A, B: SELECT bal FROM acc WHERE id = 1; -- both see 100
3 A: UPDATE acc SET bal = 150 WHERE id = 1; COMMIT;
4 B: UPDATE acc SET bal = 150 WHERE id = 1;
5 B: COMMIT;
6 A: SELECT bal FROM acc WHERE id = 1; -- 150, not 200
```

1. Start both transactions at the same level.
2. Both read 100 and each plans to add 50.

← **4 · Critical:** at Read Committed this succeeds and A's +50 is lost. At Repeatable Read it fails.

6. Repeat with REPEATABLE READ in step 1.

⚠ **Common mistake:** starting the next experiment while a session still has an open or aborted transaction. Type ROLLBACK; in both sessions first.

Part B · A Lost Update in Python Code

Two threads (“cashiers”) each add 1 to the same balance 200 times, starting from 100.

```
-- terminal on your laptop
pip install -r setup/requirements.txt
python labs/week03/python/lost_update_threads.py
-- naive: SELECT bal … then UPDATE acc SET bal = <value from Python>
-- for_update: SELECT bal … FOR UPDATE, then the same UPDATE
-- atomic: UPDATE acc SET bal = bal + 1 WHERE id = 1
```

Mode	Expected balance
naive	Far below 500
for_update	500
atomic	500

← **Critical:** the naive mode loses updates at the default level with no error. Explain why each fix reaches 500.

Deliverables

What to Hand In

One PDF report of at most three pages, submitted before the Week 4 class.

Report item	Points
E1: screenshot of the deadlock error; which session was aborted, why the other finished, and how to prevent it	2
Anomaly table from your own runs: E2, E3, E4 at Read Committed and Repeatable Read	3
E4 at Repeatable Read: why PostgreSQL showed no phantom, although the standard allows one	1
Part B: the three final balances; why naive loses updates and why each fix works	2
How an application should handle deadlock detected and could not serialize access	2
Total	10

Key point: most points are for explanations: link every observation to a concept from Lessons 3-1 to 3-3.

Papers to Read

Three required papers and three optional ones on locking, isolation levels and serializable snapshot isolation.

Before the Week 4 class

Half-page note per required paper

Required

Required Reading for Week 4

The Notions of Consistency and Predicate Locks in a Database System
Eswaran, Gray, Lorie, Traiger · Communications of the ACM 19(11), 1976, pp. 624–633

Two-phase locking and phantoms. Read the introduction and the parts on two-phase transactions and predicate locks.

A Critique of ANSI SQL Isolation Levels
Berenson, Bernstein, Gray, Melton, O'Neil, O'Neil · ACM SIGMOD 1995, pp. 1–10

Why the standard's levels are ambiguous; snapshot isolation. Read sections 1–3 and 4.2.

Serializable Snapshot Isolation in PostgreSQL
Ports, Grittner · PVLDB 5(12), 2012, pp. 1850–1861

How PostgreSQL's Serializable level works. Read sections 1–3.

Bring a half-page note per paper: the problem, the key idea, and its link to this week.

Optional

Going Further

For students who want depth, or who are choosing a topic for the course project.

On Optimistic Methods for Concurrency Control
Kung, Robinson · ACM TODS 6(2), 1981, pp. 213–226

No locks: read, then validate at commit, then write.

Serializable Isolation for Snapshot Databases
Cahill, Röhm, Fekete · ACM SIGMOD 2008, pp. 729–738

The original SSI algorithm that PostgreSQL adopted.

Generalized Isolation Level Definitions
Adya, Liskov, O’Neil · IEEE ICDE 2000, pp. 67–78

Isolation levels defined by dependency graphs, for any implementation.

Reading Questions

Try to answer each question in two or three sentences before the next class.

Q1	Why must transactions be two-phase to guarantee consistency, and why do phantoms need predicate locks rather than row locks?	Eswaran et al.
Q2	What is wrong with defining isolation levels only by the three ANSI phenomena, and which anomaly does snapshot isolation still allow?	Berenson et al.
Q3	How does SSI decide when to abort a transaction, and what does that cost compared with Repeatable Read?	Ports, Grittner
Q4	In Practice 3, E4 showed no phantom at Repeatable Read. Why does this not contradict the SQL standard?	All papers

WEEK 3 · WRAP-UP

This Week’s Work

Practice 3

Lab report

Deadlock, anomaly table and the Python lost update. Max 3 pages, 10 points.

Due: before Week 4

Quiz 3

Concurrency and ACID

Lessons 3-1 to 3-3: 16 multiple-choice questions and one short design question.

Due: before Week 4

Careers

Assignment report

Submit your careers report this week, then review three classmates’ reports.

Due: this week

Reading

Required papers

Eswaran et al., Berenson et al., Ports and Grittner. A half-page note each.

Bring to Week 4

Next week: one database becomes many. What changes when data and transactions are spread across several sites?